Password Manager Development Project

Dissertation

COMP702 - Msc Project (2024/25)

By, Abdelhadi Zaidi

(201843733)

Supervised by,

Dr. Karteek Sreenivasaiah

Department of Computer Science

University of Liverpool

Student Declaration

I confirm that I have read and understood the University's Academic Integrity Policy.

I confirm that I have acted honestly, ethically and professionally in conduct leading to assessment for the programme of study.

I confirm that I have not copied material from another source nor committed plagiarism nor fabricated data when completing the attached piece of work. I confirm that I have not previously presented the work or part thereof for assessment for another University of Liverpool module. I confirm that I have not copied material from another source, nor colluded with any other student in the preparation and production of this work.

I confirm that I have not incorporated into this assignment material that has been submitted by me or any other person in support of a successful application for a degree of this or any other university or degree-awarding body.

SIGNATURE:

DATE: November 28, 2025

Acknowledgement

This project felt like an adventure that I took. I've always wanted to work in security and understand cryptography, but I've never had the opportunity before. This project has taught me a lot about these topics, and I believe I made the right decision by choosing a topic that piqued my interest, motivating me to work harder and do my best. I'd also like to thank my supervisor, Dr. Karteek, for the meetings we've had, the majority of which were very useful, as well as the feedback I received from him and the markers on my previous submissions. What I'm hoping is that this project will help shape my future in this field or provide me with the knowledge I need to work in a career related to this industry. I enjoy research in general, and I thoroughly enjoyed this journey.

BUNKR - Password Manager

Abstract

This dissertation presents the design, development, and evaluation of a password manager that was named BUNKR, which is a standalone offline desktop password manager created for the purpose of achieving the learning outcomes of the this project. The motivation to choose this was to strengthen my technical skills, particularly in secure software development, and learn the fundamentals of what it takes to develop such software. Creating a password manager involves many tasks and challenges such as cryptography, secure data handling, interface design, and careful architectural planning, but remain sufficiently self-contained to be realistic within the overall project timeframe.

The project is focused on solving the main challenge that is to create a password manager that is usable and which applies modern encryption techniques that fit today's security standards while also remaining simple enough to implement within the given timeframe of the project. The design of the software follows a modular architecture, where cryptographic operations, user interface components, and application logic are separated from one another which made the project easier to manage and errors easier to solve. A SQLite database stores vault entries which includes passwords encrypted individually using keys derived via PBKDF2-HMAC-SHA256 and additional authentication components, including PIN verification and security-answer recovery which use Argon2id hashing. This separation between key derivation for encryption and hashing for authentication arose from practical considerations and the need to balance security with feasibility.

The implementation includes features such as password generation, clipboard auto-clear, auto-lock, duplicate and similarity detection through string distance metrics, and a recovery workflow that decrypts the master key using a secondary derived key.

Much of the development process involved learning unfamiliar technologies such as PySide6, the cryptography libraries, and database handling. As such, the progression of the software is intertwined with the development of my own technical understanding.

Evaluation consisted of functional testing, small-scale usability feedback, and a theoretical security review. Functional tests verified correct encryption, database behaviour, and recovery logic, while usability insights highlighted the importance of clarity in error messages, form validation, and visual consistency. Theoretical analysis assessed BUNKR within an offline threat model where it acknowledged the system's limitations, such as the lack of protection against device compromise, metadata leakage, etc.

The project ultimately resulted in a working application and a deeper personal understanding of secure system design as a learning outcome. Future extensions could include encrypted synchronisation, biometric unlock, browser extensions, or a more advanced backup format. The work also raises broader reflections on the challenges of building security tools responsibly, especially for developers still gaining technical confidence.

Statement of Ethical Compliance

This project falls into both data category A and participant category 0. No human participants will be involved in this project and no private/personal data will be collected. I will ensure to adhere to academic ethical guidelines and comply with relvant research regulations and policies throughout the whole duration of the project.

Table of contents:

Chapter 1

Introduction

1.1 Background and Motivation

The growing reliance on digital services has made password management a routine but critical responsibility for individuals. As users create accounts across multiple websites, applications, and online platforms, the challenge of securely storing and recalling various credentials increases significantly. According to Bonneau et al.(2012), password managers provide a practical solution by creating a secure, centralised repository for sensitive data. However, the majority of popular password managers use cloud-based services, although convenient, these systems introduce external trust dependencies, security risks, and concerns about data sovereignty.

The password manager's name, BUNKR, is derived from the word Bunker, and it arose from an interest in creating a local-first, offline password manager in which all encryption and storage takes place solely on the user's device, or bunker in this case. The motivation was not to compete with commercial tools, but rather to create a manageable learning environment in which I could be exposed to the realities of secure software development. Coming from a business-oriented academic background, the project allowed me to combine theoretical security concepts with hands-on implementation while also demonstrating to myself that I could build a multi-component application involving cryptography, databases, and user interfaces in Python.

The project also reflects a broader interest in how design decisions impact usability and security. Password managers must strike a careful balance between these two aspects

since overly complex systems may frustrate users, whereas overly simplified ones may jeopardise security. Exploring this balance became a guiding theme throughout the project, influencing choices for encryption, recovery mechanisms, interface structure, and user workflows.

1.2 Problem Statement

Despite the fact that there are many good password managers available, many people refuse to use them because they lack trust, are concerned about privacy, or are afraid of becoming reliant on the cloud. These concerns are not unfounded since there have been high-profile breaches of password-storage platforms (Anderson, 2020), and even when encryption works, metadata exposure, server misconfigurations, or compromised infrastructure can cause people to lose trust. However, manually tracking passwords or using the same ones on multiple sites is extremely dangerous for security. The primary issue this project is attempting to solve is twofold:

1. How to create and use a password manager that operates entirely offline, without the use of any external servers or cloud storage.
2. As a student that has moderate programming skills, particularly in cryptography and application architecture, how can you construct a system like this safely and logically?

Because of the the password manager is offline, all encryption, key generation, authentication, and storage must take place on the same computer and without any network communication. While this eliminates entire risk categories, it may also complicates some matters. For example, if there is no server-side verification or cloud-based fallback, recovery must be performed locally in a secure manner. Furthermore, mistakes made when dealing with sensitive data can have long-term consequences. The system must also be usable

because security alone is insufficient if the user cannot figure out how to use the interface and its features or navigate the workflows.

Another aspect of the problem is the learning process during the project. Previously, I had only done academic exercises in Python, and I had never used GUI frameworks such as PySide6 or performed cryptographic operations other than simple hashing. So the project required me to use both self-directed learning (via documentation, tutorials, online courses, and experimentation) and incremental development. This dissertation examines both the software's construction and the rationale for specific decisions, particularly in terms of reconciling optimal cryptographic practices with practical constraints such as time, experience, and feasibility.

1.3 Brief Description of the Approach

This project used an iterative, modular approach based on a layered understanding of what the system required. Instead of trying to accomplish everything at once, I divided the problem into smaller chunks, such as cryptography, data storage, interface design, session management, and recovery, and approached each as a separate learning challenge.

One of the most critical components of the program is the cryptographic layer. PBKDF2-HMAC-SHA256 is the foundation for vault encryption and It uses a unique 32-byte salt and 100,000 iterations which is a dated value by current security standards in order to generate a 256-bit key from the user's master password. The key is then used with one of three supported cyphers: AES-GCM, ChaCha20-Poly1305, or Fernet. Moreover, Argon2id hashes additional authentication data, such as PINs and recovery answers, to make it more difficult for attackers to brute-force their way into the users accounts. Separating key derivation from authentication hashing simplified threat modelling and implementation.

The data is stored in a local SQLite database, with each password entry represented as a separate row and only the password field is encrypted where as other metadata, such as titles and usernames, are not encrypted, allowing for quick searches without the need to decrypt each entry. Additionally, a different metadata file stores salts, Argon2 hashes, encryption preferences, and encrypted recovery information.

The user interface is created with PySide6 and has a simple, consistent appearance. Features such as auto-clear for the clipboard, auto-lock, duplicate and similarity detection, and contextual password generation were added over time. I changed the architecture several times throughout the project. The design eventually evolved into an MVC-style model, with the VaultManager serving as the model, GUI components as views, and the MainWindow coordinating interactions, effectively acting as the controller and supporting modules remain separate to keep things organised and manageable. This method allowed me to continue learning while making progress, and it ensured that each component could be understood, tested, and improved independently.

1.4 Outcome

The main goal of this project is to develop a desktop password manager with robust encryption, support for multiple vaults with a complete offline functionality, and a number of intuitive and user-friendly features. BUNKR can create encrypted vaults, securely store and retrieve passwords, generate passwords, detect duplicates and near-duplicates, protect the clipboard, and allow you to recover under certain conditions. The application is not intended to replicate the scale or complexity of commercial password managers; however, it effectively demonstrates the application of secure design principles in an academic project context.

The most important outcome is the knowledge gained during the development process. When I first started, I had moderate knowledge on how to create cryptographic workflows, data schemas, or a functional user interface. So the project was equally about learning how to design secure systems as it was about building the system itself.

On the other hand, there are still some existing issues, such as non-password fields being displayed in plaintext, and no automated testing or synchronisation feature. Given the educational focus of the project, these are viewed as opportunities for future development rather than deficiencies.

Chapter 2

Design

As my understanding of cryptography and application architecture deepened, the structure of the software was progressively refined to facilitate the design of BUNKR. The initial prototypes worked, but they were difficult to use because the roles were unclear and the encryption logic was too close to the interface code. Much of the design work focused on making the system more understandable and modular.

This section discusses the final architectural model, what each major component of the system does, and why certain design decisions were made. Despite its small size, BUNKR contains a variety of layers, including recovery mechanisms, graphical user interfaces, encryption, authenticated hashing, and persistent local storage. One of the project's primary challenges was to figure out how to connect these pieces in a way that made sense.

2.1 System Architecture

The architecture is fundamentally divided into five layers: application start-up layer, vault management, cryptographic functions, data storage, and the user interface. The VaultManager monitors the status of each vault and serves as the central model for encryption keys, database access, and recovery procedures. The view contains the user interface components created with PySide6. It shows information and accepts input. The interaction logic within the MainWindow acts as a controller, ensuring proper communication between the user interface and the vault logic.

This structure made it easier to add new features later, such as automatic clipboard clearing, password generation, and duplicate detection. Because each subsystem served a specific purpose, it was possible to add new features to the application without disrupting its existing components.

Architecture Diagram:

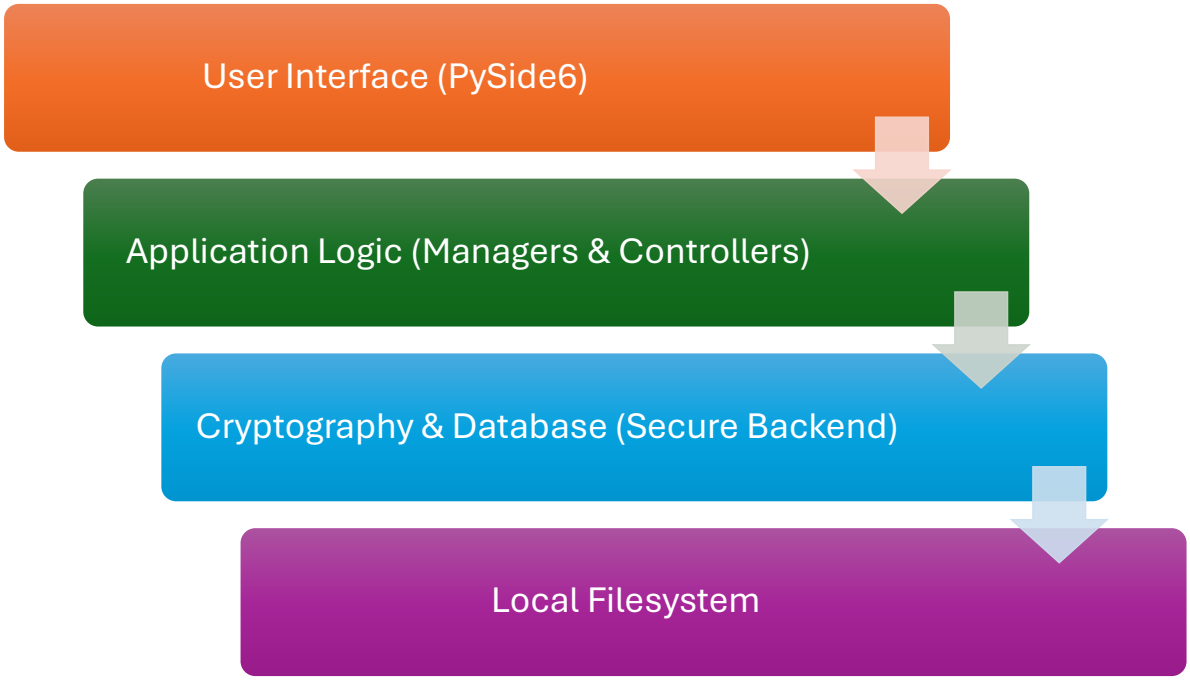


Figure 1: this diagram shows the system four-layer architecture.

2.2 Component Design and Rationale

2.2.1 VaultManager

The VaultManager became the system's focal point. It includes key derivation, encryption and decryption, metadata management, database interactions, and recovery logic. Early versions of the code performed some of these activities directly in the interface layer, but relocating them to a distinct component resulted in a cleaner and more intuitive structure. The VaultManager stores the derived master key only while the vault is unlocked, clears it when the vault is locked, and never exposes it to the user interface.

Conceptually, this component represents the application's secure core. It processes all sensitive activities and ensures that the metadata file and database are consistent. The VaultManager also handles more complicated activities, such as re-encrypting vault data when the master password changes or the user switches encryption algorithms.

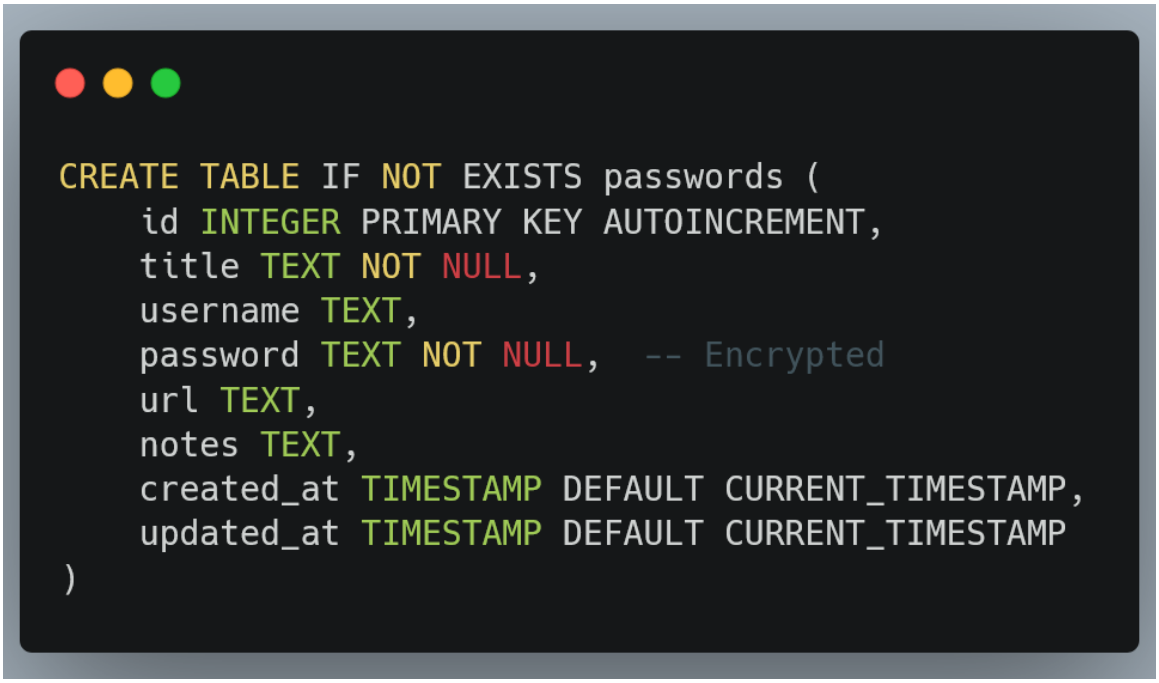
2.2.2 Metadata Structure

Alongside each vault sits a metadata file that stores cryptographic parameters and authentication data. Designing this file required understanding which information was essential for unlocking the vault and how to structure it without leaking sensitive details. The metadata includes the PBKDF2 salt, the chosen encryption method, the Argon2id hashes for the PIN and verification answer, the encrypted version of the master key used for recovery, a recovery salt, and other details such as version identifiers.

Separating the metadata from the database turned out to be beneficial for clarity as well as security. It created a clean distinction between the cryptographic configuration and the encrypted data, making it easier to reason about failure cases such as having corrupted metadata for example, and to implement consistent backup behaviour.

2.2.3 Database Schema

For storage, SQLite was chosen since it is self-contained, lightweight, and does not require a running server. This makes it ideal for an offline application with simple installation requirements. The database schema is purposefully simple as each password input is saved as a row that includes titles, usernames, URLs, encrypted passwords, notes, and timestamps.



```
CREATE TABLE IF NOT EXISTS passwords (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    username TEXT,
    password TEXT NOT NULL,  -- Encrypted
    url TEXT,
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
```

Listing 1: Database schema code.

This schema shows the hybrid storage approach where only the password field is encrypted, enabling searchable metadata while maintaining security.

A major design decision was to encrypt only the password field. In early designs, attempts were made to encrypt all fields, however this made it impossible to search or filter items without decrypting the entire database. Encrypted search would have necessitated far more advanced approaches, such as deterministic encryption or encrypted indices, which were outside the project's scope. Leaving non-sensitive metadata in plaintext provided a sensible balance between security and usability.

```
vault_data = {
        'salt': base64.b64encode(salt).decode(),
        'pin_hash': pin_hash,
        'verification_question': verification_question.strip(),
        'verification_hash': verification_hash,
        'encryption_method': encryption_method,
        'recovery_salt': base64.b64encode(recovery_salt).decode(),
        'recovery_nonce': base64.b64encode(recovery_nonce).decode(),
        'encrypted_master_key': base64.b64encode(encrypted_master_key).decode(),
        'created_at': datetime.now().isoformat(),
        'version': '1.0'
    }
```

Listing 2: Metadata structure code.

This metadata structure shows how cryptographic parameters and authentication data are organized in the JSON metadata file. The separation of metada from the database file distinguishes between configuration and encrypted data, making it easier to understand security properties and build backup systems. All sensitive values are base64-encoded to ensure JSON compatibility.

2.2.4 Cryptographic Layer

The cryptographic layer manages all procedures including key derivation, encryption, decryption, and hashing. By maintaining these functions independently from the user interface code, the likelihood of inadvertently exposing sensitive data was reduced. This layer derives cryptographic keys through PBKDF2-HMAC-SHA256, employing a 32-byte salt and 100,000 iterations to produce a vault-specific symmetric key. The system is capable of performing encryption and decryption using AES-GCM, ChaCha20-Poly1305, and Fernet, utilising Python's cryptography library. Each time an encryption is performed, a new nonce is generated and retained alongside the ciphertext. This facilitates the straightforward decryption of the entry. This layer exclusively employs Argon2id for hashing PINs and

verification answers. Due to its memory-hard nature, it is resistant to GPU based brute-force attacks, which underpin the offline threat model of the vault.

2.2.5 User Interface Design

The user interface was created with PySide6 with a minimalist and consistent design in mind. Qt's Fusion style with a custom dark palette was chosen to give the application a modern appearance while keeping the colours and visual hierarchy simple and because PySide6 was being learned while building the application, the UI went through several transformations. Early versions had unclear labels and inconsistent component spacing. Adjustments had to be made such as clearer prompts, improved dialogue layouts, and more intuitive button placement in order to increase usability.

Much of the UI design was influenced by observations rather than formal user studies. For example, error messages were rewritten several times after it was discovered that the initial versions were too technical or did not explain what users needed to do next. Even minor changes, such as adding spacing between sections or indicating when a password had been copied, improved the overall experience.

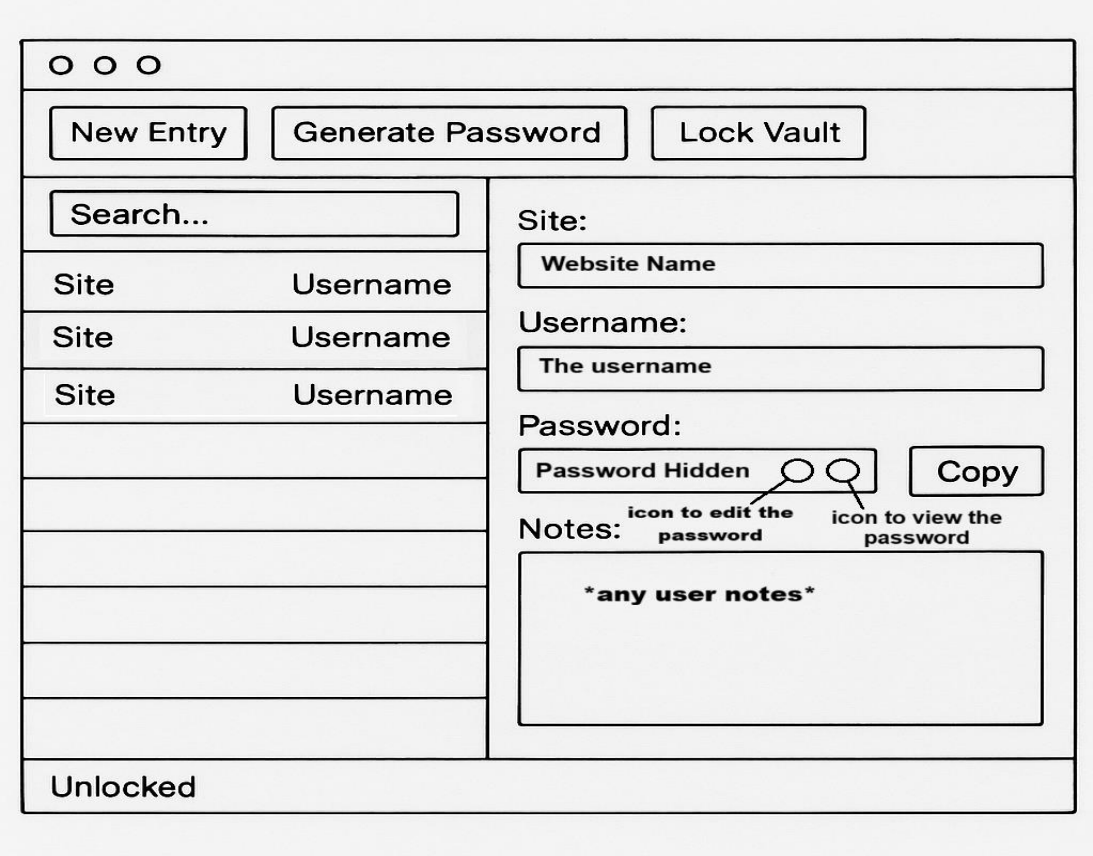


Figure 2: the initial UI/UX mockup for the password manager.

2.2.6 Password Generator and Supporting Modules

The password generator uses Python's secrets module to produce cryptographically strong passwords. It allows customisation of length, character sets, and inclusion or exclusion of ambiguous characters. The generator was initially considered as a separate window, but it was integrated as a contextual panel within the main window that can either fill a new entry automatically or replace an existing password, maintaining workflow continuity.

Other supporting modules include the clipboard manager, which deletes copied passwords after a brief time, and the auto-lock system, which locks the vault while inactive. Both features began as rudimentary prototypes and improved in reliability through repeated testing.

2.2.7 Duplicate and Similarity Detection

To discourage users from repeating or slightly modifying passwords, duplicate and similarity detection was implemented using the Levenshtein distance. This necessitated carefully decrypting existing passwords during tests and ensuring that temporary plaintext was promptly removed. Adding this feature strengthened security, as users benefit from subtle guidance towards better password habits without intrusive warnings.

Chapter 3

Implementation

The password manager implementation translated the conceptual design into a functional application. This made it the most challenging and educational phase, as it required applying cryptographic knowledge, GUI development skills, and Python programming practices simultaneously. Thus, the process involved substantial research, trial and error, and a continuous cycle of refactoring. This section documents the technical development of the system, the decisions behind critical pieces of functionality, and the practical constraints that shaped the final implementation.

3.1 Vault Creation and Initial Setup

Vault creation is the first interaction users have with the system, and implementing it required careful coordination among the interface, vault manager, and cryptographic layer. The process begins with the user entering a vault name, master password, PIN, and verification answer. Because the vault is a long-term container for sensitive data, the creation process was designed to generate all required cryptographic assets before any database files are written to disc.

The system first creates a 32-byte random salt using the operating system's secure random number generator. This salt is stored in the metadata file and is required to derive the vault's encryption key using PBKDF2-HMAC-SHA256. Early prototypes derived a key directly from the password without salting, but this was quickly corrected because it jeopardised security and contradicted the primary goal of key stretching.

```
#generate salt for key derivation
    salt = secrets.token_bytes(32)

    #setup encryption with the chosen method
    self._setup_encryption(master_password, salt, encryption_method)

    #create database with encrypted schema
    self._create_database()

    #hash the PIN using argon2
    pin_hash = self.ph.hash(pin)

    #hash the verification answer using argon2 (case-insensitive)
    verification_hash = self.ph.hash(verification_answer.strip().lower())

    #derive master encryption key for storage as recovery key
    kdf = PBKDF2HMAC(
        algorithm=hashes.SHA256(),
        length=32,
        salt=salt,
        iterations=100000,
    )
    master_key = kdf.derive(master_password.encode())
```

Listing 3: This code snippet demonstrates the vault creation process, showing how cryptographic assets are generated before any database files are written.

After generating the salt, the application uses PBKDF2 with 100,000 iterations to generate a 256-bit key. This key is the foundation for all encryption and is only stored in memory while the vault is unlocked. At this point, the metadata file is created, but authentication information must be securely embedded. The PIN and verification answer are hashed with Argon2id, which uses parameters that make brute-force attacks computationally expensive. Argon2id implementation necessitated understanding how memory-hard algorithms behave across systems and ensuring that the chosen parameters remained balanced between security and performance for most personal computers.

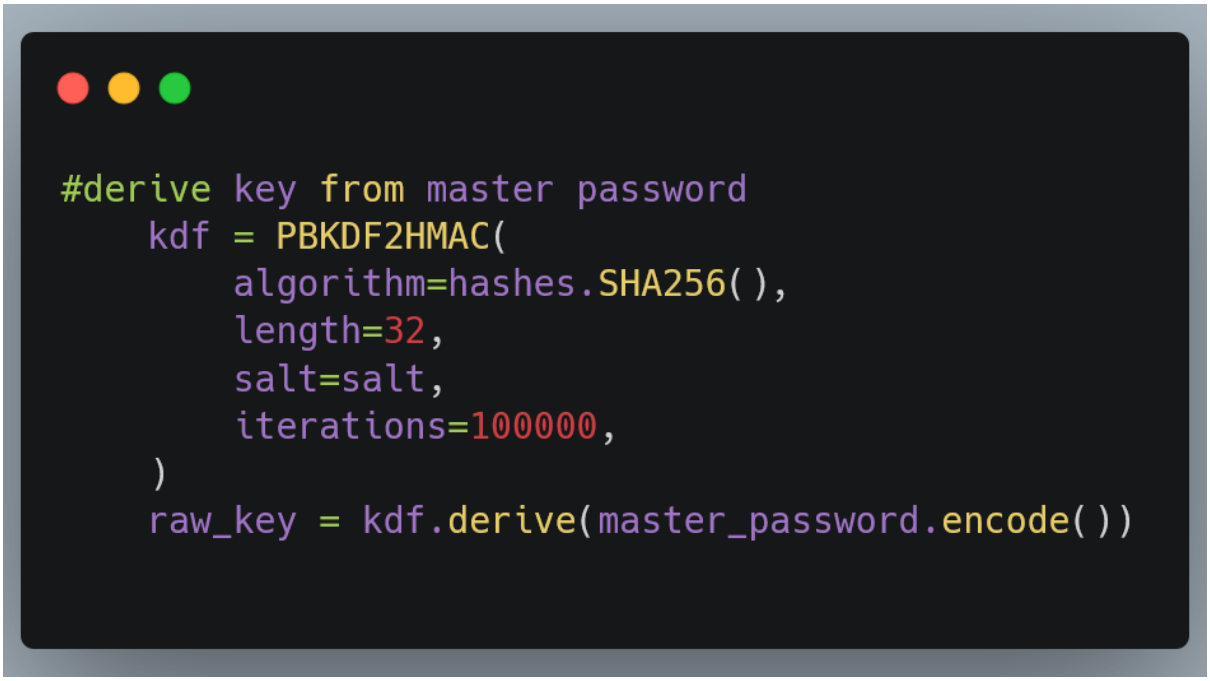
Along with the hashed values, the system keeps an encrypted copy of the master key for recovery. This necessitated creating a separate recovery key from the PIN and verification answer, and then encrypting the master key with AES-GCM. This encrypted master key, along with its nonce and salt, is stored in the metadata file. Designing this mechanism required carefully distinguishing between the concepts of "master password," "master key," and "recovery key" a distinction that was not immediately apparent at the start of the project.

Once all of the metadata has been written, an empty SQLite database with the appropriate schema is created. Only after these steps are completed does the system provide the user with confirmation that the vault was successfully created.

3.2 Key Derivation and Cryptography Implementation

Implementing the cryptographic layer necessitated extensive learning. At the start of the project, familiarity with encrypted storage theory existed, but practical experience with cryptographic APIs was limited. The Python cryptography library provided strong primitives, but properly integrating them required meticulous attention to detail.

The key derivation for vault encryption is PBKDF2-HMAC-SHA256, which was chosen because it is well-supported, reliable, and appropriate for a crossplatform Python application. Using Argon2id for vault keys was initially considered, but due to portability issues and time constraints, PBKDF2 proved to be the better option.



```
#derive key from master password
    kdf = PBKDF2HMAC(
        algorithm=hashes.SHA256(),
        length=32,
        salt=salt,
        iterations=100000,
    )
    raw_key = kdf.derive(master_password.encode())
```

Listing 4: This code shows the PBKDF2-HMAC-SHA256 key derivation process.

For encryption, three cyphers were used: AES-GCM (the default), ChaCha20-Poly1305, and Fernet. AES-GCM required explicit handling of authentication tags to ensure that the ciphertext and tag were stored together. Each encryption call generates a 12-byte random nonce, which is then added to the encrypted output and saved in the database. Decryption reverses the process by extracting the nonce, applying the same key, and verifying the authentication tag. Because GCM authentication is strict, even minor changes to the ciphertext cause decryption to fail.

ChaCha20-Poly1305 behaves similarly, but with a stream cypher instead of a block cypher. Implementing it was simple because the library abstracted much of the underlying complexity. Fernet was included primarily for legacy support and user choice, and it handled nonce and tag details internally, making it simpler but less transparent. However, because Fernet uses AES-128-CBC with HMAC-SHA256, it does not offer the same level of modern AEAD security as the other two algorithms. This reinforced the decision to use AES-GCM as the default for new vaults.

```
def _encrypt_data(self, plaintext_bytes):
        if self.encryption_method == ENCRYPTION_FERNET:
            return self.fernet.encrypt(plaintext_bytes)
        elif self.encryption_method in (ENCRYPTION_AES_GCM, ENCRYPTION_CHACHA20):
            #generate a random nonce (12 bytes for aes-gcm and chacha20)
            nonce = secrets.token_bytes(12)
            ciphertext = self.cipher.encrypt(nonce, plaintext_bytes, None)
            #prepend nonce to ciphertext: [nonce (12 bytes)][ciphertext]
            return nonce + ciphertext
```

Listing 5: this encryption method shows how various cypher modes are handled.

```
def _decrypt_data(self, encrypted_data):
        if self.encryption_method == ENCRYPTION_FERNET:
            return self.fernet.decrypt(encrypted_data)
        elif self.encryption_method in (ENCRYPTION_AES_GCM, ENCRYPTION_CHACHA20):
            #extract nonce (first 12 bytes) and ciphertext
            nonce = encrypted_data[:12]
            ciphertext = encrypted_data[12:]
            return self.cipher.decrypt(nonce, ciphertext, None)
```

Listing 6: this method extracts the nonce from the first 12 bytes and uses it to decrypt the ciphertext.

3.3 Database Interaction and Encrypted Storage

The next step after implementing the cryptographic functions was to construct the database layer. SQLite's simplicity made it ideal for an offline application, but the challenge was to keep sensitive data encrypted while metadata required for usability remained plaintext. Queries must search usernames, URLs, and titles without revealing passwords or notes stored within each entry.

When you enter a password, the system only receives the plaintext password temporarily. The VaultManager encrypts it with the current cypher and saves the encrypted output to a BLOB. The remaining fields (title, username, URL, and timestamps) are entered

directly in plaintext, allowing search operations. This hybrid approach necessitated establishing clear boundaries: a decrypted password should never be returned to the database layer or the UI without explicit user action.

```
def add_password(self, title, username, password, url="", notes=""):
        if not self.is_unlocked:
            return False

        try:
            conn = sqlite3.connect(self.vault_path)
            cursor = conn.cursor()

            #encrypt password before storing
            encrypted_password = self._encrypt_data(password.encode())

            cursor.execute('''
                INSERT INTO passwords (title, username, password, url, notes)
                VALUES (?, ?, ?, ?, ?)
            ''', (title, username, encrypted_password, url, notes))

            conn.commit()
            conn.close()
            return True
```

Listing 7: This method encrypts the password before database insertion while storing other fields in plaintext for searchability.

Retrieving entries follows a similar procedure. The UI searches the SQLite database for non-private fields, and decryption occurs only when the user selects a specific entry. The plaintext password is never saved to disc; it exists only in memory until it is copied to the clipboard or replaced by a new password while editing.

Implementing timestamps required understanding how SQLite handles date and time. The database schema uses SQLite's CURRENT_TIMESTAMP for both creation and update times, which ensures consistency across platforms. For the metadata file, timestamps were generated in Python using datetime.now().isoformat() to record when the vault itself was

created. This decision simplified unit testing by making timestamp generation predictable for metadata while leveraging SQLite's built-in functionality for database entries.

3.4 User Interface Implementation

Developing the interface in PySide6 was perhaps the most challenging learning curve. Little experience existed with Qt initially, so many features appeared gradually as understanding developed of how signals, slots, layouts, and widgets interacted. Early prototypes had cluttered windows and displayed inconsistent behaviour when switching between views. Through successive iterations, the structure was refined into a more cohesive set of windows and dialogues.

The main window displays the password table, search bar, and key interaction buttons. Dialogues are used to create vaults, unlock them, enter passwords, change settings, and recover. One of the challenges was managing window transitions, such as ensuring that unlocking a vault closed the selector window without leaving orphaned components running in the background.

Signals and slots were critical in keeping the user interface in sync with the internal state. Actions like copying a password or saving an edited entry send signals to vault-level operations. Because PySide6 runs in a single thread by default, care was required not to block the event loop with lengthy operations. Cryptographic routines are relatively quick, but re-encrypting an entire vault during a password change necessitates looping through each entry. Testing confirmed that the UI remained responsive throughout these operations, and visual cues were added to reassure users during longer tasks.

When the application starts, it uses a custom Qt palette to enable dark mode. Adjusting paddings, colours, and button styles required several attempts to achieve a consistent aesthetic. Although the interface is simple, the development process demonstrated

how much effort goes into providing a pleasant, predictable user experience, even in simple desktop applications.

3.5 Password Generator, Duplicate Detection, and Password Strength

The password generator required integrating Python's secrets module with user selected preferences. Initially, the generator lacked options beyond length, but during testing it became apparent that users might often have diverse requirements such as excluding ambiguous characters or enforcing symbol usage. Adding these controls required dynamically constructing character pools based on user input and ensuring that generated passwords contained at least one character of each chosen type.

Password strength measurement is based on entropy estimation (Ferguson, Schneier & Kohno, 2010). Although this is not a perfect representation of real-world password security, it provides helpful guidance. The formula uses the logarithmic size of the chosen character set multiplied by the password length. Implementing this calculation gave users immediate visual feedback when adjusting generator settings or editing passwords manually.

$$E = \log_2(R^L) = L \times \log_2(R)$$

Where:

E is the entropy in bits.

R is the size of the range or character pool (the number of unique possible symbols available).

L is the length of the password. (the number of characters)

$\log_2$ converts the total number of combinations into bits.

Duplicate and similarity detection required comparing the new password with all existing ones. Decrypting each stored password for comparison raised immediate security concerns, so the logic was structured to ensure that decrypted plaintext exists only fleetingly and is discarded immediately after checks complete. The similarity threshold required

experimentation since very strict thresholds yielded too many warnings, while looser thresholds allowed extremely similar passwords to pass unnoticed. The final implementation uses a balanced threshold informed by Levenshtein distance, with a default of 80% similarity to flag potentially problematic passwords.

```python
def _calculate_similarity(self, password1: str, password2: str) -> float:
        if password1 == password2:
            return 1.0

        max_len = max(len(password1), len(password2))
        if max_len == 0:
            return 1.0

        #calculate edit distance
        distance = self._levenshtein_distance(password1, password2)

        #similarity ratio: 1.0 - (distance / max_length)
        similarity = 1.0 - (distance / max_len)

        #also check if one password is a substring of another (high similarity)
        if len(password1) >= 4 and len(password2) >= 4:
            if password1 in password2 or password2 in password1:
                #calculate substring similarity
                shorter = min(len(password1), len(password2))
                longer = max(len(password1), len(password2))
                substring_similarity = shorter / longer
                #use the higher similarity value
                similarity = max(similarity, substring_similarity)

        return similarity
```

Listing 8: This algorithm calculates password similarity using Levenshtein distance and substring detection, with an 80% threshold for warnings.

3.6 Auto-Lock, Clipboard Management, and Session Handling

Security features like auto-lock and clipboard cleaning necessitated synchronising time-based events within the PySide6 environment. The auto-lock timer is refreshed with each user interaction, and if it expires, the vault closes and the keys are deleted from memory. This involved linking signals from numerous widgets to a central reset function, guaranteeing that any action, from typing to opening a dialogue, was considered as activity. The timer uses

QTimer, which integrates easily with Qt's event loop, and the default timeout is customisable through settings (defaulting to 3 minutes).

Clipboard management uses a similar timer mechanism, but implemented with Python's threading.Timer rather than Qt's QTimer, since clipboard operations need to work independently of the GUI thread. When a password is copied, the system stores the previous clipboard state, places the decrypted password onto the clipboard, and schedules a restoration after the configured delay (default 30 seconds). Initially, platform differences in clipboard ownership were underestimated, which led to inconsistent behaviour. Through testing on multiple systems, the logic was refined to make clipboard clearing reliable, using pyperclip for cross-platform clipboard access.

3.7 Error Handling and Unexpected Conditions

Error handling had to be robust because failures in cryptographic operations or file access could have serious consequences. Structured try-except blocks were used for critical functions like unlocking, re-encrypting entries, and loading metadata. When decryption fails due to an incorrect key, the system removes all sensitive data from memory and instructs the user accordingly.

One of the most important lessons learnt during this phase was understanding failure modes. Corrupted metadata files, missing nonces, and mismatched authentication tags all manifest differently, and the system must respond appropriately. Where recovery is impossible, such as when metadata files become unreadable, the application displays clear messages explaining what happened and how to restore from a backup.

3.8 Backup, Export, and Import Mechanisms

Vaults are intentionally easy to export and import. Instead of performing partial exports or custom formatting, the system simply allows the user to copy the database and

metadata files to a specified location. Importing checks for the presence of both files, confirms that they belong together, and prevents them from overwriting existing vaults by appending a "_backup" suffix to imported vault names. Implementing this simplicity helped to reduce unnecessary cryptographic risk.

Chapter 4

Experimental Results

Testing the application was essential to understanding not only whether BUNKR worked as intended but also whether its security assumptions and design decisions held up in practice. And because the project did not involve human participants or the collection of personal data, the evaluation focused primarily on functional testing, security oriented testing, usability reflective testing, and validation against the original requirements of the project. This phase revealed a number of issues some were technical and others were conceptual which guided iterative improvements and strengthened the final application.

4.1 Functional Testing

Functional testing had the objective to ensure that each component of the program functioned properly under typical conditions and this was done slowly, with each component tested individually before combining them. Early tests focused on the vault generation process, ensuring that salts were generated correctly, PBKDF2 consistently produced keys of the correct length across platforms, and metadata files were properly structured.

Incorrect password scenarios were also tested to verify that vault unlocking failed gracefully without exposing sensitive material.Password entry operations were tested using a sequence of controlled actions. Creating entries with typical and edge case inputs including long passwords, empty fields, unusual characters, helped verify that the database schema handled storage robustly. Scenarios involving rapid entry creation, deleting entries, and repeatedly editing the same item were tested to ensure that timestamps updated accurately and that encryption/decryption remained consistent. During these tests, a few early issues were encountered where plaintext passwords were briefly printed to the console during

debugging. Identifying these helped reinforce stricter separation between sensitive operations and general logging.

The user interface underwent a similar process. Each dialog had to load correctly, validate fields, and return cleanly to the main view without leaving orphaned windows or ghost processes. Recovery flows, in particular, required careful attention: incorrect PINs, incorrect verification answers, missing metadata components, and mismatched salts all needed to produce clear, user-friendly error messages. The functional testing process reinforced that cryptographic errors, in particular, should fail "loudly," so users are aware something is wrong rather than assuming the vault is empty or partially unlocked.

Step	Process	Details
1	User Input	Master password, PIN, and recovery answer are entered.
2	Validation	PIN format (6 digits) checked; required fields verified.
3	Cryptographic Setup	Generates 32-byte random salt; derives master key using PBKDF2-HMAC-SHA256 (100k iterations).
4	Database Creation	Initializes new SQLite database and password table schema.
5	Authentication Hashing	PIN and recovery answer are hashed using Argon2id.
6	Recovery Setup	Derives recovery key; encrypts (wraps) master key using AES-GCM.
7	File Writing	Writes metadata (.meta) and encrypted database (.db) files.
8	Vault Ready	Vault successfully created and ready for use.

Table 1: This table outlines the eight sequential steps in vault creation, from user input validation through cryptographic setup to final file writing.

4.2 Security Focused Testing

Security testing was the most critical aspect of evaluation, given that the project's purpose is to secure sensitive data. Unlike commercial penetration tests, this evaluation was limited to methods achievable in an academic setting, but it nonetheless uncovered several weaknesses that were later addressed.

A significant portion of the security evaluation involved verifying that key derivation and encryption behaved as expected. PBKDF2 results were tested across multiple devices to ensure the same password and salt reliably produced identical keys, while different salts produced completely different keys even when the password was identical. This confirmed that the key derivation process behaved correctly and resisted precomputed attacks.

AES-GCM's authentication behaviour was also tested by manually modifying bytes within an encrypted password entry. As expected, decryption failed immediately, raising an exception and demonstrating that tampering cannot go undetected. ChaCha20-Poly1305 responded similarly. Fernet's built-in HMAC verification also performed correctly, rejecting modified data.

To assess brute-force resistance, incorrect password attempts were simulated using an automated script. PBKDF2's 100,000 iterations introduced a measurable delay, which confirmed that large-scale brute-force attempts would be impractical. Argon2id demonstrated strong resistance as well; even a simple six-digit PIN could not be brute-forced quickly due to the algorithm's memory hardness. During this testing, it was also verified that BUNKR correctly triggered lockout behaviour and moved to recovery mode after five repeated failed attempts.

Another focus of security testing involved verifying that sensitive data was not written to disk or logs unintentionally. By observing system behaviour with monitoring tools and

scanning temporary directories, it was confirmed that decrypted passwords remained only in memory and were cleared during vault lock, program exit, or after clipboard restoration. This testing helped validate that the application adhered to the "no plaintext at rest" principle.

4.3 Usability Evaluation

Although the project did not involve formal user studies, informal usability testing was conducted by interacting with the system repeatedly throughout development. As someone not originally trained in software engineering, heavy reliance was placed on observing behaviour as a user and identifying points of confusion or friction. This self-evaluation shaped many refinements.

For example, early versions of the interface made it unclear when a vault was actually locked, causing situations where it was believed to be secured when it was not. Adding clear confirmation messages and automatically redirecting to the vault selector fixed this. Similarly, warnings during duplicate or near-duplicate password detection were initially overly opaque, referring to "low Levenshtein thresholds," which are meaningless to most users. These messages were rewritten in clearer language, but the technical details remained hidden within the underlying system.

The password generator has also undergone significant evolution in response to usability considerations. Originally, it was considered as a separate dialog, which would have interrupted workflows. Its integration as a contextual panel within the existing entry form improved the system's overall feel. The entropy indicator, which was previously displayed as a number value, was changed with descriptive categories such as "Strong" or "Very Strong," making it more understandable.

These refinements reinforced a recurring theme to ensure the coexistence of usability and security. Features that are technically secure but difficult to understand would not serve the user effectively.

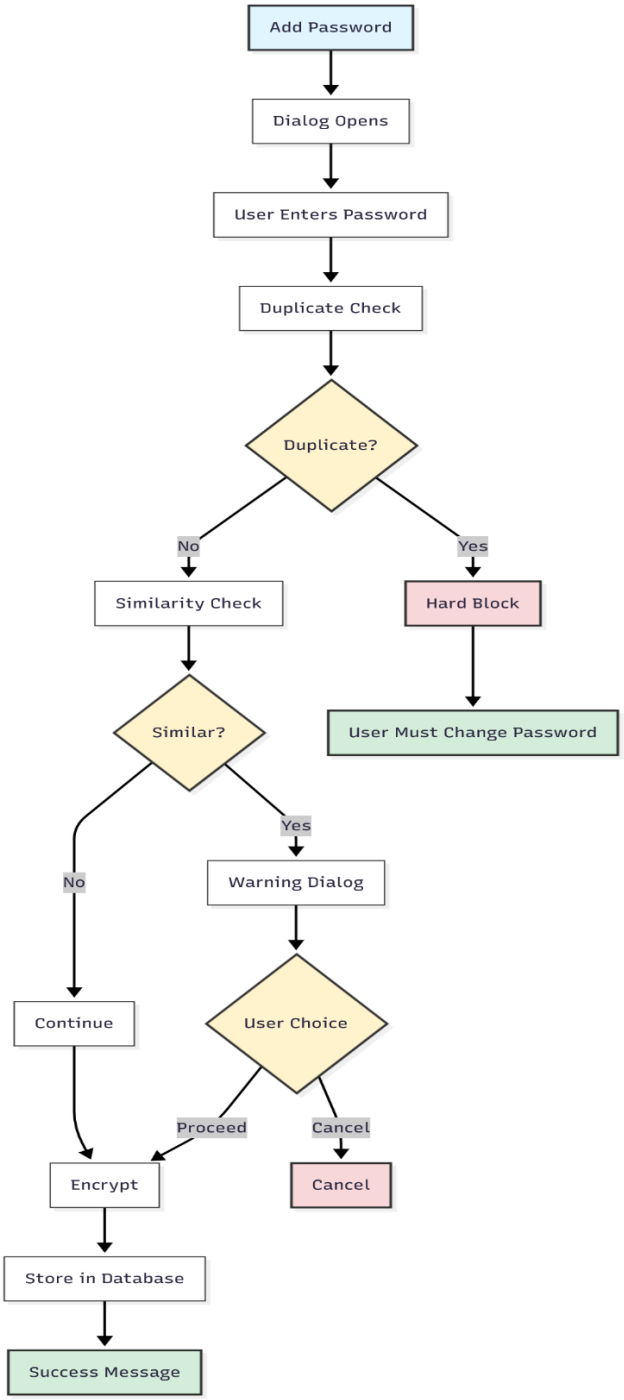


Figure 3: This diagram illustrates the password entry workflow with duplicate and similarity detection checks before encryption and storage.

4.4 Evaluation Against Requirements

Evaluating BUNKR against the proposal's initial specifications revealed that the finished system met all core objectives. The application correctly stores passwords offline, encrypted, using robust contemporary encryption methods. Multiple vault support was added, allowing users to keep their personal, work, and experimental vaults distinct without risk of key or metadata contamination. The cryptographic model uses PBKDF2 for deriving vault keys and Argon2id for hashing PINs and verification answers, fully matching the planned security architecture.

Usability requirements were met through a clear interface, dark theme, search functionality, clipboard auto-clear, and auto-lock. Security features including duplicate and similarity detection improved users' password hygiene practices. Duplicate passwords are hard blocked, preventing re-use, whereas similar passwords provide warnings that allow users to proceed if they wish. When the master password was forgotten, the recovery processes worked well and allowed vault access using only the PIN and verification answer. The password manager performance also met expectations. On average hardware, encryption and decryption were quick, and database searches ran smoothly even with a large number of records. Re-encryption operations after password changes or algorithm migrations were slower, but still adequate for an offline desktop application. During this evaluation, areas where the system could be expanded were identified.

These include optional cloud-free synchronisation using local networks, more extensive logging, and support for biometric authentication. Although these features were beyond the scope of this project, identifying them helped frame reasonable pathways for future development.

4.5 Reflection on Testing Limitations

While the testing method was rigorous for the project's scope, there were some practical limitations. There were insufficient resources and knowledge to perform advanced penetration testing, formal verification of cryptographic correctness, or fuzzing of input validation pathways. Multi-platform testing was restricted to platforms that were personally accessible, primarily Windows. Performance testing outside of the baseline benchmarks was also limited. Lastly, despite these constraints, the testing approach met objectives by focussing on validating accuracy, anticipating potential security problems, and ensuring that the program operated predictably under practical usage.

Chapter 5

Theoretical Analysis

Although BUNKR is a practical software project, its foundations rest on theoretical principles from cryptography, security engineering, and data protection. This chapter examines those principles and demonstrates how they shaped the system's design. It also outlines the theoretical limits of the system, clarifies what threats it can and cannot defend against, and situates BUNKR within established security models for encrypted storage applications.

5.1 Threat Model

A password manager is only meaningful when considered in relation to the threats it is designed to withstand. Chellu (2025), describes a Python-based password manager that uses encrypted storage and supports strong password creation. From the outset, a threat model consistent with offline, local-first password vaults was adopted. The primary assumption is that attackers do not have live control over the user's machine while the vault is in active use. Instead, the most likely threat involves an attacker obtaining the vault files and attempting to decrypt them offline.

This offline attack scenario is frequent in real-world breaches, in which stolen laptops or corrupted backups allow attackers to access encrypted data. Under this threat paradigm, securing the encryption key takes priority. The system must ensure that the encryption key cannot be deduced from the metadata, and that the master password, PIN, and verification answer are inaccessible even to the developer.

Because BUNKR is an offline tool, it does not defend against active malware on the user's device, screen capture attacks, or keyloggers. It also assumes physical access can result in stolen hardware, which makes strong key derivation essential. Recognising these boundaries helped avoid overestimating the system's guarantees and ensured that design choices remained grounded in realistic expectations.

5.2 Cryptographic Rationale for Key Derivation

The use of PBKDF2-HMAC-SHA256 for deriving the vault's encryption key was grounded in both practical and theoretical considerations. According to Dworkin (2017), PBKDF2 is still a NIST-approved function that is commonly used in secure systems such as file encryption utilities, Wi-Fi authentication, and other cryptographic protocols. The theoretical strength of PBKDF2 stems from its iterative design: each iteration slows key derivation, greatly increasing the cost of brute-force attacks.

```
kdf = PBKDF2HMAC(
        algorithm=hashes.SHA256(),
        length=32,
        salt=salt,
        iterations=100000,
    )
    raw_key = kdf.derive(master_password.encode())
```

Listing 9: This code implements PBKDF2 key derivation following NIST recommendations, balancing security and performance with 100,000 iterations.

Using 100,000 iterations strikes a mix between security and usability. A larger repetition count would improve security slightly, but it would significantly slow down unlock

operations on older hardware. The extended runtime may deter users from using stronger passwords. In contrast, a lower count would decrease security against offline brute-force attacks. The chosen setup is thus consistent with widely known security recommendations for consumer-level encryption and meets the requirements of a Python-based application.

Furthermore, the addition of salts strengthens theoretical resilience to precomputation attacks. A salt ensures that even identical passwords generate distinct derived keys, preventing attackers from utilising rainbow tables or a single computed hash across several vaults. This attribute is crucial to current password based key derivation and was included from the beginning of the design.

5.3 Authentication Hashing and Memory Hardness

While PBKDF2 sufficed for key derivation, Argon2id was selected for hashing the PIN and verification answer. The decision is rooted in the theoretical structure of memory-hard algorithms. Argon2id requires a substantial amount of memory to compute, making it resistant to GPU-accelerated brute-force attacks (Argon2 RFC, 2016), which is a critical property for defending low entropy inputs like a six-digit PIN.

From a theoretical standpoint, the use of memory hardness slows down custom hardware attacks dramatically. ASICs and GPUs thrive on parallelism, but Argon2id constrains them by forcing each attempt to store and manipulate large blocks of memory. This design choice complements the vault's threat model: offline attackers may attempt to brute-

force low-complexity credentials, and Argon2id slows these attempts significantly.

```
#hash the PIN using argon2
    pin_hash = self.ph.hash(pin)

    #hash the verification answer using argon2 (case-insensitive)
    verification_hash = self.ph.hash(verification_answer.strip().lower())
```

Listing 10: this code uses Argon2id to hash authentication factors, providing memory-hard resistance against GPU-accelerated attacks.

This division of roles (PBKDF2 for deriving the encryption key, Argon2id for hashing local authentication factors) reflect a practical balance that aligns with current guidance in cryptography research and avoids relying exclusively on one algorithm for multiple purposes.

5.4 Justification for Encryption Algorithms

Selecting encryption algorithms required examining theoretical properties, performance considerations, and the ecosystem of available cryptographic libraries. AES-GCM was chosen as the default cipher because of its AEAD (Authenticated Encryption with Associated Data) design. In theory, AEAD modes provide strong guarantees not only of confidentiality but also of integrity. The authentication tag generated by GCM ensures that ciphertext cannot be altered without detection, a property essential for resisting tampering in offline storage scenarios.

ChaCha20-Poly1305 was included as an alternative because of its favourable performance characteristics on systems without AES hardware acceleration. Theoretically, ChaCha20 offers high diffusion and uniform randomness, while Poly1305 supplies a fast one-time authenticator. Its security proof has been analysed thoroughly, and its adoption in TLS

1.3 and modern secure messaging gives confidence in its real-world reliability (Langley et al., 2015).

Fernet, while older and less theoretically robust than AEAD ciphers, was included to preserve backward compatibility and serve as a pedagogical reference point. Fernet internally combines AES-128-CBC with HMAC-SHA256, but its lack of built-in nonce management and its reliance on CBC mode place it below AES-GCM and ChaCha20-Poly1305 in the modern hierarchy. Keeping Fernet optional allowed users migrating from legacy vaults to adopt BUNKR without discarding existing data, while still encouraging secure defaults for new vaults.

```
elif encryption_method == ENCRYPTION_AES_GCM:
        self.fernet = None
        self.cipher = AESGCM(raw_key)
        self.key = raw_key
    elif encryption_method == ENCRYPTION_CHACHA20:
        self.fernet = None
        self.cipher = ChaCha20Poly1305(raw_key)
        self.key = raw_key
```

Listing 11: this code initializes AEAD ciphers (AES-GCM or ChaCha20-Poly1305) from the same derived key, providing both confidentiality and integrity.

5.5 Security Boundaries and Assumptions

No security model is absolute, and recognising boundaries is essential for accurate theoretical analysis. BUNKR assumes that the user's device is not compromised during active usage. If malware records keystrokes or screen data, no level of encryption can meaningfully mitigate this. Similarly, side-channel attacks targeting Python's memory handling or the

operating system's clipboard implementation fall outside the scope of the application's protections.

Another boundary stems from the nature of password-based key derivation. While PBKDF2 makes brute-force expensive, a weak master password remains theoretically vulnerable. This limitation motivated the inclusion of password strength indicators and duplicate detection features, but ultimately the strength of the vault still depends on user behaviour.

BUNKR also cannot defend against corruption of both the metadata file and its backup. The separation of metadata and database files strengthens modularity and security, but it means that losing the metadata file renders the vault unrecoverable. This is not a flaw but a theoretical result of using password derived encryption keys: without the salt and configuration parameters stored in metadata, key derivation cannot be reconstructed.

5.6 Theoretical Justification for Recovery Mechanism

Designing the recovery mechanism required examining theoretical constraints on secure key storage. The master key, once derived from the master password, must not be stored in plaintext under any circumstance. However, recovery requires that the master key (or a key-encrypting-key) be retrievable when the user forgets their password.

To resolve this, the theoretical model of "key wrapping" was applied, where the master key is encrypted using a separate recovery key derived from the PIN and verification answer. The security of this mechanism rests on three principles:

1. The recovery key must be derived from independent authentication factors.
2. The encryption used to wrap the master key must include integrity protection.
3. The recovery process must never reveal the master key unless both factors are verified.

AES-GCM satisfies the second requirement by providing integrity verification through its authentication tag. Argon2id satisfies the first requirement by ensuring that both the PIN and verification answer are stored only in hashed form and cannot be reversed. The recovery workflow itself satisfies the third requirement by verifying hashes before decrypting the wrapped master key.

```
#create recovery key from PIN + verification answer for encrypting master key
    recovery_salt = secrets.token_bytes(32)
    recovery_kdf = PBKDF2HMAC(
        algorithm=hashes.SHA256(),
        length=32,
        salt=recovery_salt,
        iterations=100000,
    )
    recovery_key_material = f"{pin}{verification_answer.strip().lower()}".encode()
    recovery_key = recovery_kdf.derive(recovery_key_material)

    #encrypt master key with recovery key using aes-gcm
    recovery_cipher = AESGCM(recovery_key)
    recovery_nonce = secrets.token_bytes(12)
    encrypted_master_key = recovery_cipher.encrypt(recovery_nonce, master_key, None)
```

Listing 12: this code implements key wrapping for recovery, encrypting the master key with a recovery key derived from PIN and verification answer.

This structure provides a theoretically sound approach to recovery that avoids insecure shortcuts such as backdoors or plaintext key storage.

5.7 Theoretical Assessment of Usability& Security Trade-offs

Throughout development, several theoretical tensions between usability and security were encountered. According to Nielsen (1993), usability must coexist peacefully with security. For instance, fully encrypting all database fields theoretically maximises confidentiality but drastically reduces usability by preventing search functionality. Conversely, leaving certain metadata fields unencrypted theoretically lowers confidentiality but makes the system more practical for real world use.

Balancing these trade-offs required considering established security models that prioritise protecting the most sensitive assets while allowing metadata necessary for functionality to remain accessible. This approach aligns with similar design choices in widely used password managers, where titles and usernames may be stored unencrypted while passwords themselves remain encrypted.

The decision to include clipboard auto-clear and auto-lock features reflects a theoretical understanding that users often unintentionally expose plaintext through their own actions. These features therefore compensate for gaps in human behaviour, not just technical systems.

5.8 Positioning BUNKR Within the Wider Cryptographic Landscape

The theoretical assumptions used in the password manager are consistent with current practices in secure software. The use of PBKDF2, Argon2id, and AEAD encryption reflects the recommendations of highly regarded cryptographic researchers and institutes. Although the project is academic and the implementation is not meant for commercial use, adhering to recognised theories assures that BUNKR does not inadvertently rely on old or unverified methodologies. At the same time, the project does not aim to innovate by developing new cryptographic algorithms or unique storage paradigms, which would be theoretically unsound. Instead, it reconstructs well-known principles in a coherent, instructive manner. This method exhibits an understanding of the theory while avoiding frequent mistakes in amateur cryptography design.

Chapter 6

Conclusion and Future Work

The development of BUNKR provided an opportunity to become extensively involved in both the theoretical and practical elements of secure software engineering. What began as a relatively simple idea to create an offline password manager grew into an in-depth study of cryptography architecture, secure storage, and user-centred interface development. This final section examines the project as a whole, assesses its performance in meeting the specified objectives, and highlights potential areas for future improvement.

6.1 Project Summary and Reflection

The project set out to design and implement a standalone, offline desktop password manager that prioritised user privacy and strong security practices. Through the application of established cryptographic techniques, modular architectural design, and iterative development, BUNKR successfully delivers these objectives. The system securely stores sensitive information using PBKDF2 based key derivation, modern AEAD encryption schemes, and Argon2id hashing for local authentication factors, all aligned with the threat model established at the beginning of the project.

Working on the password manager required dealing with a number of challenges from the start, including balancing security principles with usability constraints, efficiently designing a multi-layered Python application, and addressing the practical limitations of GUI frameworks and cryptographic libraries. It was important to consider the complexities of integrating encryption logic with interface components, as well as how minor design choices can result in security risks. Through research, experimentation, and repeated refactoring, these challenges became valuable learning experiences that shaped the final system.

Implementing the recovery mechanism, duplicate password detection, and auto-lock features, in particular, helped me better grasp how theoretical notions translate into user-friendly security. These components demonstrated that secure systems are not just about cryptographic strength, but also about anticipating human error, interaction patterns, and realistic threat scenarios.

6.2 Evaluation of Achievements

BUNKR achieves its primary goal providing a functional, safe, and user-friendly offline password manager based on modern cryptographic concepts. The system encrypts data reliably, protects users from frequent errors, and clearly separates encrypted and plaintext components. It accomplishes these objectives without relying on external services or cloud infrastructure, hence maintaining the primary design principle of full user ownership.

Beyond reaching technical accuracy, the project successfully demonstrates how a whole software system can be constructed from an initial notion to a finished product. Iterative refinement made the architecture more modular, the implementation more robust, and the user experience more consistent. These accomplishments reflect a significant improvement in both technical proficiency and conceptual comprehension.

6.3 Recognised Limitations

No software project is without limitations, and acknowledging them is important for accurate interpretation of the system's security and applicability. BUNKR does not provide protection against malware, physical device compromise, or sophisticated side-channel attacks. Browser integration, biometric authentication, and device synchronisation are not included, despite being typical features in commercial password managers.

The system also depends heavily on users choosing strong master passwords. Although password strength indicators and duplicate detection encourage better practices, users still retain responsibility for the ultimate strength of their vault. Additionally, the design requires careful backup of both the metadata and the database files; losing the metadata file makes recovery impossible due to the nature of key derivation.

From an implementation perspective, while functional testing was done throughout the development process, a formal automated test framework would increase robustness and maintainability. These deficiencies do not jeopardise the project's main learning objectives, but rather highlight areas where future work could improve the system's production readiness.

6.4 Future Work

Several avenues for future development emerged throughout the project. One promising direction involves enabling synchronisation without compromising the offline-first principle. This could be approached by supporting local network sync, encrypted file replication, or integration with self-hosted storage solutions under full user control. Such features would broaden BUNKR's applicability while preserving the privacy guarantees that define its identity.

Expanding usability features is another natural step. Adding biometric unlock options, for example, could make the system more accessible without weakening security if implemented correctly. Browser extensions or auto-fill capabilities would also improve practicality, though implementing them securely would require additional research and careful design to avoid exposing decrypted information.

From a technical standpoint, future work could include more extensive logging, automated regression tests, and a more formal error-handling framework. Improving the codebase with type hints and documentation would also support long-term maintainability,

especially if the system were ever extended by other developers. Implementing more advanced encrypted search techniques, such as deterministic encryption or encrypted indexing, could boost usability while maintaining confidentiality, but this would greatly increase project complexity.

Finally, additional theoretical research could look into formal verification of cryptographic components, evaluation under more advanced threat models, or the incorporation of hardware-based security features. These enhancements would transform the system from an instructional project to a more sophisticated security tool.

6.5 Final Remarks

BUNKR ultimately demonstrates that secure software design is attainable at the postgraduate level with diligence, research, and a willingness to learn through experimentation. The project not only resulted in a functioning application but also deepened understanding of cryptography, software architecture, and usability-driven design. The process reinforced that secure systems rely on careful alignment between theory and implementation, and that user behaviour must be considered as seriously as algorithmic strength.

Finally, as a research project, BUNKR is a coherent, secure, and well-designed system that represents significant personal growth in terms of technical competence and confidence in dealing with complex software challenges. This progress will help to shape future projects, whether in security, software engineering, or other fields at the intersection of technology and business.

Chapter 7

BCS Project Criteria and Self Reflection

This final section evaluates BUNKR against the six learning outcomes defined by the British Computer Society (BCS) and presents a personal reflection on the development process. Although the project is technical in nature, it also represents a learning journey marked by challenges, discoveries, and the progressive development of skills that were unfamiliar at the outset. The BCS criteria provide a structured lens through which to articulate this development and assess how the project aligns with expected standards for a postgraduate computing qualification.

7.1 Alignment with BCS Criteria

7.1.1 Application of Practical and Analytical Skills

Throughout the project, practical software engineering skills were applied, including Python programming, database design, GUI development, cryptographic implementation, and structured testing. Many of these were new at the outset and had to be built from tutorials, documentation, and academic sources. Analytical skills were especially important when comparing cryptographic algorithms, evaluating threat models, structuring recovery workflows, and identifying safe ways to combine usability with security. Implementing PBKDF2, AES-GCM, ChaCha20-Poly1305, and Argon2id required understanding both theoretical and applied aspects of cryptography, not simply copying examples. Each part of the system depended on careful reasoning about data flow and risk.

7.1.2 Innovation and/or Creativity

Although the project does not seek to innovate at the level of new cryptographic algorithms, it does demonstrate creativity in system design. The recovery mechanism, for instance, required creatively combining established cryptographic concepts in a way that was usable and secure. Similarity detection and password hygiene features blended algorithmic thinking with practical concerns about user behaviour. The system's architecture evolved through creative restructuring as understanding developed of what worked and what did not. The design ultimately reflects thoughtful creativity within the constraints of established security models.

7.1.3 Synthesis of Information, Ideas, and Practices

Completing the project required synthesising information from multiple disciplines: cryptography, software engineering, user experience design, and cybersecurity principles. This synthesis is evident in choices such as storing only the sensitive password field as ciphertext while leaving metadata unencrypted for usability, or dividing authentication factors between PBKDF2 and Argon2id based on their theoretical strengths. The final implementation reflects the integration of academic reading, practical experimentation, and iterative refinement. Each design decision required combining several ideas into a coherent whole.

7.1.4 Meeting a Real Need in a Wider Context

BUNKR addresses a real need for offline, private password management which is an area often overlooked in mainstream tools that rely heavily on cloud synchronisation and centralised storage. While the application is not intended for commercial release, its core philosophy responds to legitimate privacy concerns and demonstrates how cryptographic tools can empower users to retain ownership of their data. The system's focus on offline operation, absence of telemetry, and transparent design principles positions it within a wider conversation about digital autonomy and data minimisation. Even as a learning project, it

engages meaningfully with real-world issues.

7.1.5 Self-Management of a Significant Piece of Work

Managing the project required careful planning, disciplined time management, and constant reassessment of priorities. Although the scope was underestimated at the beginning, particularly the challenges of GUI design and cryptographic integration, the process involved learning to break tasks into manageable components, track progress, and refactor when necessary. The modular architecture that emerged was not only a technical decision but also a practical strategy for managing complexity. Balancing research, implementation, debugging, documentation, and testing demanded a level of self-management that had not previously been experienced in software development.

7.1.6 Critical Self-Evaluation of the Process

Reflecting on the project reveals strengths as well as areas for improvement. Satisfaction exists with the system's security model, the clarity of the interface, and the modularity of the codebase. Adaptation to cryptographic concepts that initially appeared overwhelming was accomplished, as was persistence when debugging complex interactions between the UI and encryption logic. However, it is acknowledged that the project could have benefited from earlier structuring, more systematic testing, and the use of automated tools such as linters or test frameworks from the start. Initial unfamiliarity with PySide6 led to inefficient early prototypes, and some refactoring could have been avoided with better upfront planning. Nonetheless, recognising these shortcomings has been as valuable as the technical outcomes themselves.

7.2 Personal Reflection on the Project Journey

Developing BUNKR has been a formative experience, both technically and intellectually. Coming from a business background, the project was approached with limited

programming experience and little knowledge of cryptographic implementation. Much of the early work involved reading documentation, watching tutorials, and experimenting with code until understanding developed of how the pieces fit together. This process could be frustrating at times, but it taught how to learn complex technical material independently and how to persist through ambiguity.

One of the most important lessons was understanding the relationship between theory and implementation. Concepts such as key derivation, AEAD encryption, memory hardness, and authentication tags initially felt abstract, but implementing them in code revealed how theory constrains practical decisions. For example, key derivation choices affect performance and responsiveness, while encryption modes influence error behaviour and storage structure. Grappling with these interactions strengthened the ability to reason about systems holistically.

The project also revealed how unpredictable software development can be. Issues that seemed minor such as restoring clipboard state or handling window focus in PySide6 that sometimes required significant investigation. Conversely, components expected to be difficult, such as integrating ChaCha20-Poly1305, turned out to be straightforward thanks to high-quality cryptographic libraries. These experiences taught that confidence comes not from knowing everything in advance but from being able to learn and adapt.

Finally, BUNKR emphasised the value of usability in security applications. A secure system that users find confusing is not truly secure because confusion frequently leads to errors, which can have a negative impact on the system’s usability. Designing meaningful warnings, clear error messages, and simple workflows necessitated looking beyond cryptography and considering how users behave in practice. Thus, the balance between cryptographic strength and human factors is perhaps the project's most valuable lesson.

Appendix

Appendix A: Database schema table.

Column Name	Type	Description
id	INTEGER	Primary key (autoincrement)
title	TEXT	Entry name (plaintext)
username	TEXT	Username (plaintext)
url	TEXT	Website or system URL (plaintext)
password_enc	BLOB	Encrypted password (ciphertext + nonce)
notes	TEXT	Optional notes (plaintext)
created_at	TEXT	Timestamp generated in Python
updated_at	TEXT	Timestamp updated on modification

This schema shows the hybrid storage approach where only the password field is encrypted, enabling searchable metadata while maintaining security.

Appendix B: cryptographic and system libraries used for encryption, hashing, and database operations.

```
import os
    import shutil
    import sqlite3
    import json
    import secrets
    from datetime import datetime
    from cryptography.fernet import Fernet
    from cryptography.hazmat.primitives import hashes
    from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
    from cryptography.hazmat.primitives.ciphers.aead import AESGCM, ChaCha20Poly1305
    from argon2 import PasswordHasher
    from argon2.exceptions import VerifyMismatchError
    import base64
```

These imports show the core cryptographic and system libraries used in the vault manager. The cryptography library provides all encryption functionality, argon2 handles password hashing for PINs and verification answers, and standard library modules (sqlite3, secrets, json) handle database operations, secure random generation, and data serialization.

Appendix C: PySide6 components used for building the graphical user interface.

```
from PySide6.QtWidgets import (
        QMainWindow, QWidget, QVBoxLayout, QHBoxLayout, QGridLayout,
        QPushButton, QLineEdit, QTextEdit, QLabel, QTableWidget,
        QTableWidgetItem, QHeaderView, QMessageBox, QDialog, QDialogButtonBox,
        QCheckBox, QSpinBox, QComboBox, QGroupBox, QSplitter, QFrame,
        QMenuBar, QMenu, QStatusBar, QProgressBar, QTabWidget, QFileDialog, QInputDialog
    )
    from PySide6.QtCore import Qt, QTimer
    from PySide6.QtGui import QAction, QIcon, QFont, QPixmap, QKeyEvent
```

These imports demonstrate the comprehensive use of PySide6 (Qt6) for the graphical user interface. The widgets module provides all UI components (buttons, text fields, tables, dialogs), QtCore handles core functionality like timers and event handling, and QtGui manages actions, icons, and fonts. This modular import structure reflects the extensive GUI capabilities required for the application.

Appendix D: Levenshtein Distance Implementation.

```python
def _levenshtein_distance(self, s1: str, s2: str) -> int:
        if len(s1) < len(s2):
            return self._levenshtein_distance(s2, s1)

        if len(s2) == 0:
            return len(s1)

        previous_row = list(range(len(s2) + 1))
        for i, c1 in enumerate(s1):
            current_row = [i + 1]
            for j, c2 in enumerate(s2):
                insertions = previous_row[j + 1] + 1
                deletions = current_row[j] + 1
                substitutions = previous_row[j] + (c1 != c2)
                current_row.append(min(insertions, deletions, substitutions))
            previous_row = current_row

        return previous_row[-1]
```

This implementation of the Levenshtein distance algorithm uses dynamic programming to calculate the minimum number of single-character edits (insertions, deletions, substitutions) needed to transform one string into another. The algorithm is optimized to use O(min(len(s1), len(s2))) space by processing rows iteratively.

Appendix E: Password Strength Calculation.

```python
def calculate_entropy(self, password: str) -> float:
        charset_size = 0
        password_lower = password.lower()

        if any(c.islower() for c in password):
            charset_size += 26
        if any(c.isupper() for c in password):
            charset_size += 26
        if any(c.isdigit() for c in password):
            charset_size += 10
        if any(c in self.symbols for c in password):
            charset_size += len(self.symbols)

        if charset_size == 0:
            return 0

        import math
        return len(password) * math.log2(charset_size)
```

This entropy calculation demonstrates how password strength is measured theoretically. The algorithm identifies which character types are present in the password and calculates the entropy based on the size of the character set and password length. This provides users with objective feedback about password strength, helping them make informed security decisions.

Appendix F: Offline Password Manager Comparison Table.

Feature / Capability	**BUNKR**	**KeePassXC**	**KeePass (Classic)**	**PasswordSafe**
Core Operation Model	Fully offline, local-first	Offline by default	Offline by default	Offline, local database
Default Encryption Method	AES-GCM (256-bit)	AES-256 (AES-CBC + HMAC)	AES-256 (AES-CBC + SHA-256)	Twofish-256
Alternative Ciphers	ChaCha20-Poly1305; Fernet (legacy)	None (supports plugins)	None (plugin community exists)	None
Key Derivation Scheme	PBKDF2-HMAC-SHA256 (100k iterations)	Argon2 + AES-KDF	AES-KDF	Password-based key derivation (configurable)
Local Authentication Factors	Master password + PIN + recovery answer	Master password only	Master password only	Master password only
Recovery Mechanism	Encrypted master-key recovery (PIN + answer)	No built-in recovery; reset = data loss	No recovery; forget = data loss	No recovery; forget = data loss
Duplicate Password Detection	Yes	No	No	No
Similarity Detection (Levenshtein)	Yes (warns for similar passwords)	No	No	No
Password Generator	Customisable (entropy scoring, secure RNG)	Yes (complex generator)	Yes	Yes
Clipboard Auto-Clear	Yes (default 30s)	Yes	Yes (plugin-based)	Yes
Auto-Lock Timeout	Yes (configurable)	Yes	Yes	Limited (OS-dependent)

Feature / Capability	BUNKR	KeePassXC	KeePass (Classic)	PasswordSafe
Vault File Structure	SQLite + metadata file	Single .kdbx file	Single .kdb file	.psafe3 file
Multiple Vault Support	Yes (isolated vaults)	Yes	Yes	Yes
UI Technology	PySide6 / Qt	C++/Qt	Windows Forms (.NET)	Native C++
Ease of Use	Beginner-friendly, clean UI	Moderate complexity	Technical, utilitarian	Simple but limited
Customisable Security Settings	Yes (cipher choice, clipboard timers)	Strong but technical	Strong but technical	Moderate customisation
Cloud Sync	None (offline-only by design)	Optional through third-party tools	Optional via plugins	None built-in
Platform Availability	Windows, Linux (potentially macOS)	Windows, macOS, Linux	Windows only	Windows, macOS, Linux (ports vary)
Ideal User	Users wanting private, offline storage with strong UX	Technical users comfortable with advanced configs	Highly technical users	Users wanting simplicity over features

References

Acar, Y., Fahl, S. & Mazurek, M., 2016. *You Get Where You're Looking For: The Impact of Information Sources on Code Security*. IEEE Symposium on Security and Privacy.

Anderson, R., 2020. *Security Engineering: A Guide to Building Dependable Distributed Systems* (3rd ed.). Wiley.

Argon2 RFC, 2016. *RFC 9106 – The Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. Internet Engineering Task Force.

Bonneau, J., Herley, C., van Oorschot, P. & Stajano, F., 2012. *The Quest to Replace Passwords: A Framework for Comparative Evaluation of Web Authentication Schemes.* IEEE Symposium on Security and Privacy.

Dworkin, M., 2007. *NIST Special Publication 800-38D: Recommendation for Block Cipher Modes of Operation – GCM and GMAC*. National Institute of Standards and Technology.

Dworkin, M., 2017. *NIST Special Publication 800-132: Recommendation for Password-Based Key Derivation*. National Institute of Standards and Technology.

Ferguson, N., Schneier, B., & Kohno, T., 2010. *Cryptography Engineering: Design Principles and Practical Applications*. Wiley.

Krawczyk, H., Bellare, M., & Canetti, R., 1997. *HMAC: Keyed-Hashing for Message Authentication.* RFC 2104.

Langley, A., Hamburg, M. & Turner, S., 2015. *ChaCha20 and Poly1305 for IETF Protocols*. RFC 8439, Internet Engineering Task Force.

Nielsen, J., 1993. *Usability Engineering*. Morgan Kaufmann.

OpenSSL Project, 2023. *OpenSSL Cryptography and SSL/TLS Toolkit Documentation*. Available at: https://www.openssl.org/

Peterson, Z., 2018. *Authenticated Encryption: AES-GCM and Beyond*. Communications of the ACM, 61(11), pp. 104–113.

Python Software Foundation, 2023. *Python 3.x Documentation*. Available at: https://docs.python.org/

PySide6 Documentation, 2023. *Qt for Python Official Documentation*. The Qt Company.

Saltzer, J. & Schroeder, M., 1975. *The Protection of Information in Computer Systems*. Proceedings of the IEEE.

The Cryptography Library, 2023. *cryptography.io Documentation*. Available at: https://cryptography.io/en/latest/

Weir, M., Aggarwal, S., Collins, M. & Stern, H., 2010. *Testing Metrics for Password Creation Policies by Attacking Large Sets of Revealed Passwords.* ACM Conference on Computer and Communications Security.

Chellu, R. (2025). *Design and implementation of a secure password management system for multi-platform credential handling*. ResearchGate. https://www.researchgate.net/publication/392897064_Design_and_Implementation_of_a_Secure_Password_Management_System_for_Multi-Platform_Credential_Handling

Corey Schafer (2020). PyQt5 Tutorial – Signals and Slots. YouTube [Online Video]. Available at: https://www.youtube.com/watch?v=Vde5SH8e1OQ

Programming with Mosh (2019). Python and SQLite – Database Programming for Beginners. YouTube [Online Video].
Available at: https://www.youtube.com/watch?v=byHcYRpMgI4

KeePassXC Project (2024). KeePassXC User Guide and Documentation. Available at: https://keepassxc.org/docs/

KeePass Password Safe (2024). Official Documentation and Feature Overview. Available at: https://keepass.info/help/base/

PasswordSafe Team (2024). PasswordSafe Official Documentation. Available at: https://pwsafe.org/